# “Make Math Easy” MME Programming Language Final Report

Daisy Aptovska, Andrew Herman, Krisha Patel, and Mariami Shinjiashvili

Computer Science Program, The Pennsylvania State University at Abington

CMPSC 470: Compiler Construction

Professor Craig A. Nelson

May 4, 2026

**Language Whitepaper**

## Executive Summary

In mathematics, there are thousands of complex calculations that are made to reach one result. Once students have learned the innerworkings of a mathematical methodology, they move on to being engineers, scientists, physicists, etc. However, these working professionals no longer need to complete complex operations to learn the technique of solving specific types of equations as they have already acquired these skills. MME is a Python-interpreter-based language specifically designed for natural language understanding in basic algebra, calculus, and statistical problems. Non-programmer individuals will now have a coding language that they have access to that does not have a complex learning curve with difficult syntax. MME will be a powerful tool in all industries requiring mathematics, and it will provide ease-of-use and efficiency.

## Abstraction

Mathematics is often a challenging subject that takes many complex calculations. Software tools have developed around mathematics, and it is crucial that programming is accessible for all. While many programmers and students are well-versed in higher level mathematics, “Make Mathematics Easy” (MME hereafter) strives to provide a solution for non-programmers providing technical professionals outside of technology an easy methodology for complex calculations. MME will follow a basic natural-language-like structure that allows for non-programmers to solve accurate and efficient basic algebra, calculus, and statistical problems.

## Introduction

In current mathematic software, we have access to powerful tools that perform instantaneous computations due to modern computing with tools such as MATLAB, other high-

level programming languages, and even calculators. However, MME hopes to provide an efficient, word-based methodology for understanding mathematical equations, allowing uses to directly type phrases along with their equations to receive a step-by-step solved solution.

The Problem Statement

- **Accessibility:** Modern-day high-level programming languages still require prior knowledge of the languages syntax, making the power of modern-computing and programming less accessible to those who aren't technically versed. MME allows an AI-like accessibility to all individuals in need to efficient calculations with simple understanding and use of the MME programming language.
- **Interpretability:** Using a compiler is very efficient because of speed; however, there is lack of line-by-line debugging. Using an interpreter-based language as the intermediate code generation tool for MME provides interpretability where users can analyze bugs in their calculations line-by-line.

Background and Motivation

Over the last century, computers have had astounding breakthroughs, gained popularity, and have been integrated into daily life. For some background, high-level programming languages have gained popularity since the 1990s. They hold powerful programming abilities with ease-of-use for programmers with many built-in functionalities. Through the last few years, generative AI has gained immense international popularity, completely changing the workflow of both technical and non-technical working professionals, students, educators, and essentially everyone.

However, there are two key factors high-level programming languages and generative AI lack. High-level programming languages may have ease-of-use for skilled programmers, and newcomers still may require weeks to adapt to languages. Generative AI is still on the rise. It tends to make mistakes ("hallucinations"), and in high-risk situations such as healthcare or engineering, calculative mistakes are detrimental to the workflow.

With MME, the language will help eliminate these concerns. Users will quickly adapt to the natural-language-like syntax. Concerns for mistakes will be eliminated since MME will not use an AI-approach for calculations, but it will be backed with intermediate code generation by Python, an interpreter-based, high-level programming language that provides accurate calculations.

## Core Language Features

The core features of MME include the following: basic algebra, calculus, and statistics functions. MME will use a natural-language-like syntax and a Python-based intermediate code generation to run these features. MME have efficiency from these features due to the interpreter-based nature. MME will allow users to type code statements (with specific syntax) with their MME code to run within the MME system (based in Python).

## The Type of System - Static Type System or Non-Static System

This MME system is a non-static system since it does not compile, is interpreter-based, and creates instances of objects as they are created from the line-by-line nature of MME. MME's benefit is speed since it creates the tokens, translates, and immediately provides results; it does not need a executable file since it is not a compiled language.

## Example of Syntax and Semantics

For syntax, MME will follow natural language based on how mathematical problems are commonly formatted. Using natural language for the syntax of MME assists with MMEs easy-of-use. The syntax will follow a structure as such: *keyword phrase operation + colon separation + equation/expression*. For example, the following statement would be syntactically valid is MME: “Solve for x : ( 3 * x) = ( 5 * x )+ 3 ”. Another example may be the following: “Solve for y : x + 4 + y = 1”. Additionally, MME will be non-case sensitive on the keyword phrase operation, but it will be case-sensitive on the equation/expression since variables can vary.

For semantics, a crucial feature that users need to focus on in MME is that their keyword operation is placed onto the right operation. Additionally, MME users will need to ensure they have the proper input of their equation/expression. Another crucial part of semantics and syntax is the spacing. An MME code statement requires that there are spaces between symbols to ensure an accurate calculation, so even though no syntax errors stem from this, it is crucial for users to understand this rule so they receive accurate results for what they are calculating.

Compiler Architecture

For the frontend of the MME interpreter, the structure will focus on lexical analysis and syntax analysis. For the backend of the MME interpreter, the code will be fed through to Python translate MME code statements.

Comparison to Existing Languages

To compare MME to other popular programming languages, this section focuses on how MME compares in easy-of-use, productivity, and performance. As for easy-of-use, MME’s core pillar is to provide easy-of-use with natural language phrases as the syntax of the language. MME will have keywords and key phrases to describe equations and their order such as “solve

for x", "integrate for y", "derive for z", etc. Through this simple methodology, MME is far easier to use than other high-level languages such as Python, Java, or C++ because in other high-level languages, they require programmer-designed functions or classes to perform more complex multi-step operations.

As for productivity, MME hopes to provide productivity based on its ease-of-use. Users of MME will be able to quickly pick up the skills to use MME since the syntactical pattern of MME follows a natural language pattern. Users will not need to spend hours learning the syntax of the language, built-in functionalities, and practicing their coding abilities to utilize MME. While high-level languages like Python, Java, and C++ are powerful languages, they take several weeks to learn or even months for users who have never programmed before.

Lastly, for performance, MME will be as efficient as possible for an interpreter. While interpreters are slower by nature than compilers, the efficiency of Python is still quite powerful, and MME will use Python as its intermediate language. Java and C++ are more efficient in performance than Python as they follow a hybrid and full compiler-based structure, but this trade-off contributes to the ease-of-use of MME as users will have the power of line-by-line debugging.

Summary of the Benefits of the Language

MME holds many benefits for working professionals. Automating mathematics has been a long process since the beginning of computing, but simplifying this process for non-technical individuals has been difficult. Generative AI has been known to struggle with mathematics; therefore, efficient and accurate mathematical computing is not accessible for non-technical individuals. MME will provide easy-of-use, efficiency, and simplicity. Easy-of-use is essential

for a programming language, so users can adapt quickly and understand the syntax and semantics of the language.

Vision for the Future of the Language

For the future of MME, the development of visualizations and even more complex calculations would be a great addition. Additionally, adding more keywords to MME will help develop the complexity of the natural language it can understand. Adding the functionality of sentence structure ("subject verb object" structure) can help with creating even more ease-of-use for the language. For MME supporting basic operations and understanding in its early development, in the future, it would be excellent to incorporate different subject areas of mathematics that support more complex operations and visualizations to assist working professionals.

**Language Specification Document**

Introduction

In modern-day computing, there are thousands of tools available to programmers, however, there are no programming languages that are usable "out of the box" for non-technically skilled individuals. All programmer languages take time to learn the syntax, understand the structures, and overall master. With MME natural language capabilities, this programming language will eliminate the need for a long training period on a language, and working professionals such as engineers, scientists, and physicists will no longer need to spend tedious hours solve manual calculations. MME is the first natural language calculator programming language.

Lexical Grammar

MME's lexical grammar is a crucial component to the main syntactical structure of the language. Tokens will be broken up into keywords, colon separators, and terms and operators of an equation/expression. Some keywords are solve for, derive for, etc. "Solve" can be a token. "Derive" is another token. "x" may be another token. This structure will help MME break down the MME code to its core components, and based on the arrangement of these lexical tokens, MME can move onto the next steps of processing the code.

Syntactic Grammar and Structure

MME's syntax is simple, and it has 3 major components. It will contain the natural language phrase/keyword first, the colon separator second, and the equation/expression for the operation to be performed on last. Essentially, MME's goal is to be able to read some problems

from an algebra or calculus textbook, translate them into code, and solve them step-by-step while being backed by a Python interpreter.

Semantics Structure

Due to MME's nature of being backed by a Python interpreter, MME will need users to ensure that they are following the semantic rules for proper calculation. Since interpreters go line-by-line rather than checking statically like compilers, MME will provide results at every line. The semantics of MME require that all keywords are on the left of the colon separator with all numbers, variables, and operators are on the right side of the colon operator. Additionally, all symbols must include spaces in between them to ensure they are properly tokenized, thus ensuring the correct tokens are fed to the translator for calculation.

Standard Library

The standard library will include the following basic operations:

- Addition, subtraction, multiplication, and division
- Modulus, exponentiation, and irrational number definition

MME's standard library will include built-in keyword support. In future works, MME may have more libraries implemented into the language that support visualization, statistical operations, and more complex equation solving. We used Python's sympy library as our main extension into our standard library.

Implementation-Specific Details (optional)

MME will be implemented using Python as the language the MME language is coded in, using Python classes to parse, define, translate, and execute the MME code. Within this Python code, it will also catch MME syntactically errors through Python's built-in error catching.

Examples and Rationale

MME will be created to support most in arithmetic, algebra, and basic calculus. Below, there is a list of syntactically valid code statement examples that MME will be able to solve:

Solve for x : ( 2 * x ) + 1 = ( -1 * y ) / 3

Solve for y : ( 2* x ) + 1 = ( -1 * y ) / 3

Derive for x : y = ( 2 * x ) ^ 3 + ( 3 * x ) ^ 2 + 5x + 1

Logically, these are important code segments for MME to be able to understand as the goal of MME is to understand natural language in solving a mathematical problem. In future works, a possible library implementation may be to show hints or more detailed explanations for steps of solving. The basic original version of MME will simple be able to understand and thus compute the output from the code segments.

The Structure of Statements for the language

The structure of the statements for the language follows its syntax and semantic structure. The syntax must follow the 3-component rule of MME (keyword/phrase, separation operator, mathematical expression/equation).

The Reserved Words of the Language

Due to MME's natural-language-like syntax, there will be many reserved words in MME. As of now, MME hopes to include the following keywords and key phrases for the first launch of

MME: solve for <variable name here>, derive for <variable name here>, integrate for <variable name here>, simplify, average, mode, max, and min.

The Data Types in the language

MME will have 3 main data types towards the syntax of the language. MME will have the keyword/phrases type, the separation operator type, and the mathematical equation type. Within these 3 types, there will be sub-datatypes. In the keyword/phrase type, there will be single-word keywords and multi-word phrases that describe the operation to be performed on the equation. There will be the separation operator which will be defined as a colon. Lastly, there will be the equation/expression data type in which there will be numbers, letter variables, and operators.

The Arithmetic Operations permitted in the language

Due to MMEs scope, the following arithmetic operations will be permitted with the ability to expand upon them for complex calculations such as derivatives and integrals:

- Addition, subtraction, multiplication, and division
- Modulus, exponentiation, and irrational number definition

The Comparative Operators in the language

MME does not support the typical comparative operators. While they are not crucial to the essence of MME, they will be helpful if users need to solve inequalities or compare answers of their functions. Due to time constraints, our equations that can be solved require "=".

The Selection Sequences are capable of the language

Selection sequences will not be supported by MME instructions in the current version. MME operates based on individual code statements that execute a given mathematical operation, thus not requiring multiple branches of logic.

**The Repetition Sequence is capable of the language**

Repetition sequences will not be supported by MME. Mathematics, up until a calculus I course, typically does not require the complexity of loops, and it will help keep MME simple for non-programmer users.

The Procedures, Functions, and or Methods permitted in the language

MME does not have the ability to write functions. In future works, this would be a great implementation to allow users to write custom functions that are visualized or more complexities added. However, due to the nature and case-of-use of MME, they are not necessary for the basic functionality.

Appendices and index

There are no figures or word definitions in this section that require an appendix or index.

**Language Tutorial**

In our language tutorial, we chose to make the installation and use of MME quite simple to create ease-of-use for previously non-programmer users. To utilize MME, simply type “git clone https://github.com/daisyapto/CMPSC470_MME” into your terminal. This will clone the entire MME language to your local machine. From here, simply run the “MME.py “Python script, and you will be prompted with an MME terminal. From here, there are three main options users can type into the terminal: an MME code statement, “test mode”, and “exit”. When an MME code statement is ran (with proper syntax), the result is instantly returned. If there are syntax errors, the terminal prints an error message. When “test mode” is entered, the MME language runs an internal test of sample code statements (3 of every type of function that we implemented. When “exit” is entered, the MME language exits its execution.

The ReadMe file in our project repository includes more details on documentation and simple general steps of testing and utilizing MME. The following sample code statements are examples of the proper syntax to use in the MME language. A crucial note is that every symbol requires that there is a space in between them to return the proper, accurate result; this is an essential syntax rule of MME.

The essential MME code reference guide (also used in “test mode”):

1. “Solve for x : y = x + 1”
2. “Solve for y : x = ( y + 1 ) ^ 2”
3. “Solve for z : y = z – 3”
4. “Simplify : y = x + 1 + 2”
5. “Simplify : z = y + x + 1 + 3 + 5”

6. “Simplify : x = y + 1 + 3 + 3 + ( 5 - 3 ) ^ 2”
7. “Derive for x : y = x ^ 2 + 1”
8. “Derive for y : x = y ^ 3 + ( 2 * y ) + 1”
9. “Derive for z : y = z ^ 2 + 1”
10. “Integrate for x : y = x ^ 2 + x + 1”
11. “Integrate for y : x = y + y ^ 2 + 1”
12. “Integrate for z : y = z ^ 2 + 1”
13. “Average : [ 3 , 5 , 7 ]”
14. “Average : [ 8 , 190 , 3 , 6 , 7 , 3 , 7 , 8 ]”
15. “Average : [ 19.5 , 3.5 , 3.2 , 4.8 , 4.4 , 9.99 ]”
16. “Mode : [ 3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6 ]”
17. “Mode : [ 3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5 ]”
18. “Mode : [ 9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6 ]”
19. “Max : [ 3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6 ]”
20. “Max : [ 3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5 ]”
21. “Max : [ 9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6 ]”
22. “Min : [ 3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6 ]”
23. “Min : [ 3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5 ]”
24. “Min : [ 9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6 ]”

**Language Reference Manual**

Our language reference manual is in 5 main sections: solving, simplifying, deriving, integrating, and statistical calculating. To solve equations, use “solve for x” (or any other variable), and MME will specifically find the variable that needs to be solve for. To simplify, MME follows a simple methodology for this. The function will use the sympy library’s simplify function to reduce an expression to its simplest form. To derive, the methodology is simple. The input must be an expression, not an equation, and it must include one variable for no syntax errors. To integrate, a similar structure and code statement must be used. For statistical calculations, a list of numerical values must be on the right side of the separation colon, using a Python-like syntax for the list, such as “[ 3 , 5 , 7 ]” to ensure a proper, error-free calculation. Once again, an essential syntax rule of MME for all these calculations is that every symbol has a space between it. This ensures proper tokenization of your code statement. A lack of spaces between your symbols will cause a semantic error, meaning the MME code statement will execute, but it will return inaccurate results. The following is a list of all our keywords/phrases:

- “Solve for <variable name here>”
- “Derive for <variable name here>”
- “Integrate for <variable name here>”
- “Simplify”
- “Average”
- “Mode”
- “Min”
- “Max”

**Project Development Plan**

In our project development, there was a methodological structure followed. Once the team was formed, ideas for the language were discussed, and the team choose to follow an idea original drafted by a team member. From here, the team discussed the preliminary development by determining the main idea/concept of the language, the structure, the syntax, and translation concepts. Then, we developed the frontend stages of the interpreter such as the tokenizer and symbol table. After developing these aspects, we moved to developing the parser. We additionally created some introductory test suites during frontend development. From here, we moved to backend development (translation). Lastly, we completed an automated test suite for the language that runs basic, syntactically-correct code statements and shows the output process in “test mode”. We also had test team members complete manual testing (simple unit, integration, and system tests) on the symbol table. This symbol table is not directly used in our final development, but it served as an introductory testing tool. Our translator can understand our tokens directly.

**Language Evolution**

The MME language begins with simplicity. Our initial goals and versions include the following: ease-of-use, efficiency/speed, and accuracy of calculations. We chose to use Python to implement MME as it allows for an interpreter-based language with the leveraging of Python libraries. Additionally, it allows for line-by-line execution which is a core principle of MME. To design the language, we began with creating the front-end interpreter components, starting with a tokenizer and a symbol table. These elements allow for the code statements to be broken down and organized for translation. To begin translation, we created a functions file that allows for the functions to be called based on the symbols in the code statements and their values. The translator uses these functions to translate the token that has the keyword for the function that needs to be executed, calls the function, and thus translates the code. This allows for the output to be calculated and returned. Lastly, we complete our MME development through a testing suite that tests in three major components: unit, integration, and system testing. We created some manual tests by creating sample MME code statements, and we also created some test suites that allow for the code to be test automatically, stating that the tests were passed based on calling the testing module. Our initial testing included manual testing of three types (unit, integration, and system) by our testing team members. Our final testing includes an automated testing module that runs 3 code statements for each type of function, tokenizes, translates, and executes the results, showing a full descriptive testing module when MME executes “test mode”. In future works, MME could evolve even further into more versions. We could implement more complex calculations, allow for visual mathematical graphing, and create even more tests to further develop MME’s abilities.

**Translator Architecture**

Our translator follows a simple architecture to accurately create results. Using our tokens produced by the tokenizer, the translator extracts the keywords/phrases, and it determines the function that needs to be called and the key variable. From here, the string data is passed into the sympy library in Python, which changes the type of the tokens from string data to sympy object. These symbols, operators, numbers, and equations are then formatted into the structure of the sympy library, solved for, and the results is returned to the user. This translator architecture is essential in the functionality and efficiency of MME. We initially tested with using a symbol table; however, due to the use of this external library, we found that using the tokens directly helps in condensing the steps in passing the data.

**Component Developers**

To develop the components of MME, we separated our functionalities into separate Python files to create proper MME code modularization. We started with a tokenizer which is the basic first step of an MME code statement execution. From here, we initially created a symbol table, which was the second step of MME and was used in initial testing. However, in final development, we transition to using a functions module and a translator module directly after tokenization. We used the tokens directly into the translation step, and the translation stpe pulled from our pre-defined functions created with the sympy library in Python. Our translator created and returned the final calculated result. Lastly, we created an automated test suite that tests the different functions, as components, in MME. Additionally, this test suite, using the “test mode” command, is useful for users understanding the syntax of MME.

**Development Environment and Runtime**

The development environment for MME follows a simple set-up. Users of MME should have an installation of Python 3.x since the backbone of MME utilizes and leverages Python functionality and external library usage. Additionally, users should install the required libraries in the “requirtments.txt” file available in our repository. To run, the user must execute the MME.py file, and they will be prompted with an MME terminal where MME code statements can be entered. To enter automated testing of pre-chosen code statements, users can see the MME process by entering “test mode” into the terminal. To exit MME, type “exit” into the MME terminal.

**Test Plan**

For the MME programming language, we did testing in multiple stages (initial and final) to ensure that each part of the system works correctly and that all components function together. In our initial testing, for unit testing, each module was tested separately. The tokenizer was tested by using different mathematical and keyword-based expressions to check it correctly splits input into tokens and identifies keywords, helpers, variables, operators, numbers, and separators. The symbol table was tested by checking individual operations such as inserting entries using enterEntry(), getting values using getEntry(), editing entries using editEntry(), and deleting entries using deleteEntry(). This confirmed that each component works correctly on its own before combining them. For integration testing, the tokenizer and symbol table were connected to ensure that tokens generated from an expression could correctly be identified and stored in the symbol table. For example, expressions like “Solve for y : g = y + 9” were tokenized and then passed into the symbol table, where variables, operators, and numbers were correctly stored. For syntax parsing testing, the tokenized input was checked to see if it follows the correct structure of MME language rules, such as having a keyword at the beginning, “Solve” or “Simplify”, followed by a valid expression format. Test cases included correct expressions like “Simplify : ( x + 3 ) ^ 2”. For the translator stage, tokenized and validated input was tested for correct transformation into structured symbol table entries. This step checked that mathematical expressions are converted into a format that can later be optimized. Lastly, system testing was performed by running the full MME program with multiple complete test inputs. This included solving and simplifying expressions with integers, floats, variables, and parentheses. The system was tested end-to-end to ensure that the tokenizer, symbol table, syntax checking, and translation

all work together without errors. Overall, during initial testing, the system successfully processed inputs and confirmed that the system overall works as expected.

During final testing, we tested the entire system since the translator was completed, and we were receiving final output results. We created a test suite that tests every MME functionality (solving, simplifying, deriving, integrating, and statistical calculation) by creating three code statement tests, tokenizing them all, translating them, and outputting results of each step. We found that this was a step to fine-tune our syntax, ensuring that these code statements have proper syntax to show how MME should ideally perform. We also included some error catching when the syntax is incorrect. We found how these errors show up during testing; it is important to specify the variable for solving, deriving, and integrating, utilizing for keyword. We also found that our syntax is present, meaning our main rule of spacing all symbols is crucial to get proper output. Overall, our system functions well during final testing, and automatic testing can be completed with the phrase “test mode”. To exit MME, type “exit” into the MME terminal, and this feature works well too during testing. While MME has sensitive syntax, when follow closely, the system functions well according to our testing.

## Conclusions

In the development of MME, our team collaborated to develop an interpreted language, MME, backed by Python. We implemented an MME terminal to display a line-by-line methodology of the execution of code statements. We colloboarded as a team, and we created excellent results for the first version of MME. Throguh our development process, we followed a systematic approach, assigned tasks to within the team, and developed a tested product. Overall, we found MME to perform well. In future works, MME can be expanded upon by adding new features. Features such as visualization, advanced calculus, function writing, and more are excellent mathematical features to include into MME.

**Code Listing**

Our code for MME is accessible at the following link: https://github.com/daisyapto/CMPSC470_MME. This repository includes additional details to install, run, and test are included in the ReadMe file. In the documentation folder in our repository, our “Tutorial_and_Reference.pdf” document, which is based on our Language Tutorial and Language Reference Manual sections, shows how to utilize the language syntax and solve example practice problems. Lastly, our documentation folder also includes this report, which provides all details necessary for the implementation of MME.

**References**

ankthon. (2022). Python | Getting started with SymPy module. GeeksforGeeks. https://www.geeksforgeeks.org/python/python-getting-started-with-sympy-module/.

Hierso, S. (2022). *Coding From 1849 to 2022: a Guide to The Timeline of Programming Languages.* IEEE Computer Society. https://www.computer.org/publications/tech-news/insider-membership-news/timeline-of-programming-languages

Imarchit19. (2025). *Difference Between Compiler and Interpreter*. GeeksforGeeks. https://www.geeksforgeeks.org/compiler-design/difference-between-compiler-and-interpreter/

Nelson, C. A. (n. d.) *Creating a White Paper.pdf.* Canvas.

Nelson, C. A. (n. d.) *Example of a Completed Programming Language Specification Document.pdf.* Canvas.

Nelson, C. A. (n. d.) *An Example of White Paper Proposal In Compiler Construction For A New Programming Language.pdf.* Canvas.

Nelson, C. A. (n. d.) *Language Specification Documentation Review.pptx.* Canvas.

Nelson, C. A. (n. d.) *Patricia Zero Specifications – Draft.pdf.* Canvas.

*What are AI Hallucinations?* (n.d.) IBM. https://www.ibm.com/think/topics/ai-hallucinations

Werner, J. (2024). *AI Is Usually Bad At Math. Here's Why It Matters.* Forbes. https://www.forbes.com/sites/johnwerner/2024/10/07/ai-is-usually-bad-at-math-heres-what-will-happen-if-it-gets-better/

Asishmishra0595. (2025). Memory Management in Python. GeeksforGeeks. https://www.geeksforgeeks.org/python/memory-management-in-python/

Palaksinghal9903. Semantic Analysis in Compiler Design. GeeksforGeeks. https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/